

Design Document (DD)

An Energy-Efficient Dual Hydroponic System for Organic/Inorganic Capsicum Chili Farming

Project ID: 25-26J-035

**Individual component: Low-Power, Resilient
Communication System for Hydroponic Monitoring**

Galappaththi D.S

IT22355782

**BSc (Hons) Computer Systems & Network
Engineering**

Supervisor: Ms. Shashika Lokuliyana

1/11/2026

TABLE OF CONTENTS

1. Introduction	4
1.1 Purpose.....	4
1.2 Scope.....	4
1.3 Definitions, Acronyms, and Abbreviations	4
1.4 Overview	5
2. Overall Description.....	6
2.1 Product Perspective.....	6
2.1.1 System Interfaces	7
2.1.2 User Interfaces	7
2.1.3 Hardware Interfaces	7
2.1.4 Software Interfaces	7
2.1.5 Communication Interfaces	8
2.1.6 Memory Constraints.....	8
2.1.7 Operations	8
2.1.8 Site Adaptation Requirements.....	9
2.2 Product Functions	9
2.3 User Characteristics	9
2.4 Constraints	10
2.5 Assumptions and Dependencies	10
2.6 Apportioning of Requirements	10
3. Specific requirements	11
3.1 External interface requirements	11
3.1.1 User Interfaces	11
3.1.2 Hardware Interfaces	11
3.1.3 Software Interfaces	12
3.1.4 Communication Interfaces	12
3.2 Architectural Design	13
3.2.1 High-Level Architectural Design.....	13
3.2.2 Hardware and Software Requirements with Justification.....	14
3.2.3 Risk Mitigation Plan with Alternative Solutions	14
3.2.4 Cost-Benefit Analysis for the Proposed Solution	15
3.3 Performance Requirements	15

3.4	Design Constraints	16
3.5	Software System Attributes.....	16
3.5.1	Reliability.....	16
3.5.2	Availability.....	16
3.5.3	Security	17
3.5.4	Maintainability.....	17
3.6	Other Requirements	17
4.	Supporting information.....	19
4.1	Appendices.....	19
5.	References	22

1. INTRODUCTION

1.1 Purpose

The purpose of this document is to define a complete, implementable design for the embedded networking subsystem and its integration with the backend services. It provides the architecture, module designs, data formats, interfaces, constraints, and evaluation approach required for Progress Presentation 1 (PP1).

1.2 Scope

This document covers the design of the data ingestion and intelligence pipeline for the dual hydroponic system. It focuses on:

- (i) Sensor data publishing from ESP32 nodes to Firebase Realtime Database (RTDB) using Wi-Fi
- (ii) UDP Gateway ingestion path that offloads cloud communication from ESP32
- (iii) Flask-based ML server that reads sensor data from Firebase and writes predictions back
- (iv) Interfaces that allow the dashboard to visualize current values, history, and predictions.

1.3 Definitions, Acronyms, and Abbreviations

Term	Meaning
ESP32	Low-power Wi-Fi/Bluetooth microcontroller used as the tank node.
UDP	User Datagram Protocol; transport-layer protocol used for low overhead transmission.

Application-layer protocol over UDP	Custom payload format and behaviors implemented above UDP (packet structure, priorities, ACK rules).
QoS	Quality of Service; prioritization rules to ensure critical alerts are delivered under energy constraints.
Direct cloud ingestion	ESP32 uploads readings directly to Firebase over the internet.
GSM	Cellular communication used as a fallback path (SIM800L module).
Deep Sleep	Low-power ESP32 mode where most components power down until timer wakeup.
ML	Machine Learning; server-side models used for forecasting/analysis (e.g., energy availability, anomaly alerts).
Gateway-based ingestion	ESP32 sends UDP packet to PC gateway, gateway uploads to Firebase.

1.4 Overview

The system monitors two hydroponic tanks (Organic and Inorganic) using ESP32 sensor nodes. Sensor readings are stored in Firebase RTDB, which acts as a central data hub. A Flask ML server loads trained ML models (stored as artifacts) to predict battery behavior, recommended sleep interval, and sensor stability. Predictions are written back to Firebase so that the dashboard can display both current sensor values and ML outputs. An optional UDP gateway provides a lightweight ingestion path where ESP32 sends compact UDP packets to a PC gateway that validates and uploads the data to Firebase.

2. OVERALL DESCRIPTION

This project implements a cloud-connected dual hydroponic monitoring solution where Firebase Realtime Database (RTDB) acts as the central data hub. ESP32-based tank nodes continuously capture sensor readings for both Organic and Inorganic tanks (such as pH, EC, temperature/humidity, water level and other operational values) and publish them to Firebase through Wi-Fi. The database maintains both the latest readings and historical logs so that the system can support real-time monitoring as well as trend analysis and tank-to-tank comparison.

A server-side intelligence layer is provided using a Flask-based ML server, which loads pre-trained model artifacts and performs predictions such as battery behavior forecasting, recommended sleep interval calculation, and sensor stability assessment. The ML server reads the most recent sensor values from Firebase, executes inference, and writes prediction outputs back to Firebase for direct consumption by the dashboard/UI. In addition, the design supports an UDP gateway ingestion method where ESP32 can send compact UDP packets to a LAN gateway that validates (CRC) and forwards data to Firebase—reducing node-side overhead and providing an alternative ingestion path during development or constrained conditions.

2.1 Product Perspective

The updated system architecture is cloud-hub-centric: Firebase RTDB stores the latest sensor values, history, and ML predictions. This reduces backend infrastructure work (database management and scaling) and supports real-time dashboards. For sensor-to-cloud ingestion, two alternative paths are supported:

- (A) Direct cloud ingestion from ESP32 to Firebase via Wi-Fi internet, and
- (B) Gateway-based ingestion where ESP32 sends a small UDP packet to a LAN gateway and the gateway uploads to Firebase.

The gateway path is included to reduce node-side complexity (HTTPS/auth/retries) and to strengthen the energy-efficiency justification.

2.1.1 System Interfaces

The solution interfaces with the following system-level components:

- ESP32 Wi-Fi stack and networking APIs (for direct Firebase upload and/or UDP send).
- Server OS networking stack (Flask server and optional UDP gateway run on a PC/VM/cloud instance).
- Firebase Admin SDK (server-side authenticated access) and Firebase client access (UI).

2.1.2 User Interfaces

The primary user interface is mobile application. Key UI principles include: clear separation of Organic vs Inorganic tank views, real-time status indicators, history graphs, and a dedicated predictions/alerts panel. Detailed UI descriptions are provided in Section 3.1.1.

2.1.3 Hardware Interfaces

Hardware required for the student component:

- ESP32 node(s) connected to sensors (pH, EC, temperature/humidity, water level/distance, etc.).
- Wi-Fi router/hotspot to provide LAN + internet access.
- Optional gateway PC (laptop/mini PC) to run `udp_gateway.py`.
- Server environment (PC/VM/cloud) to run the Flask ML server (can be same as gateway during development).

2.1.4 Software Interfaces

Interfaces with other software products/components include:

- Firebase RTDB (central storage for sensor data and predictions).
- Flask ML server APIs (health + prediction endpoints tested using Postman).
- Dashboard/UI (reads Firebase to display real-time values, history, predictions).

2.1.5 Communication Interfaces

Interface	Protocol	Purpose	Notes
ESP32 -> Firebase (direct)	HTTPS	Upload current values/history	Requires auth and cloud connectivity
ESP32 -> UDP Gateway (optional)	UDP (LAN)	Lightweight ingestion to gateway	Binary packet + CRC validation
Gateway -> Firebase	HTTPS (Admin SDK)	Upload decoded readings	Offloads cloud complexity from ESP32

2.1.6 Memory Constraints

ESP32 memory is constrained; therefore, direct cloud ingestion must keep payloads small and avoid excessive buffering. In gateway mode, ESP32 uses a compact binary UDP packet to reduce formatting overhead (compared to large JSON). The ML server requires memory for model loading (e.g., scikit-learn and TensorFlow/Keras artifacts).

2.1.7 Operations

Normal and special operations expected from users/admins:

- Normal: View live sensor readings, predictions, and recent alerts for each tank.
- Normal: Compare organic vs inorganic tank performance using aligned history graphs.
- Special: Start/stop ML server and UDP gateway during deployment/testing.
- Special: Configure Firebase paths/credentials and update thresholds/alert rules (admin).

2.1.8 Site Adaptation Requirements

Deployment requires site-specific configuration: Wi-Fi SSID/password, Firebase project credentials/URL, tank naming/path mapping (Organic/Inorganic), and server endpoint settings for the Flask ML server. If the UDP gateway is used, the gateway IP address and UDP port must be configured in ESP32 firmware.

2.2 Product Functions

- ESP32 publishes latest sensor data for Organic and Inorganic tanks to Firebase RTDB (direct cloud ingestion).
- ESP32 sends a compact UDP packet to a gateway which validates and uploads the same data to Firebase.
- Flask ML server reads Firebase sensor data and generates three ML outputs (battery prediction, sleep prediction, stability prediction).
- ML server writes predictions back to Firebase under prediction nodes for UI consumption.
- Dashboard reads Firebase to display current values, history, and predictions; supports comparisons and alerts view.

2.3 User Characteristics

User Type	Skill Level	Typical Tasks
Farmer/Operator	Novice to intermediate	Monitor values, check warnings, take action
Research/Evaluator	Technical	Analyze trends, compare tanks, validate predictions
Admin/Maintainer	Technical	Run servers, manage Firebase credentials, configuration updates

2.4 Constraints

- ESP32 resource constraints (RAM/flash) limit heavy libraries and large payload formats.
- Cloud dependency: direct ingestion requires stable internet access to Firebase.
- Gateway dependency (if used): gateway device must be powered on and reachable over LAN.
- Model dependency: ML predictions depend on availability of trained artifacts and consistent feature formatting.

2.5 Assumptions and Dependencies

- Firebase RTDB is available and properly configured for read/write access.
- ESP32 can connect to Wi-Fi and reach Firebase (direct mode) OR reach the gateway on LAN (gateway mode).
- The Flask ML server has access to Firebase Admin credentials (service account key).
- Sensor readings are provided in a consistent feature order expected by the ML models.

2.6 Apportioning of Requirements

Planned implementation order (PP1 to PP2):

Priority	Requirement/Module	Planned Stage
P1 (Essential)	Firebase RTDB structure for latest values + history (Organic/Inorganic)	PP1
P1 (Essential)	Flask ML server: load 3 models and expose prediction endpoints	PP1
P1 (Essential)	ML server read from Firebase and write predictions back to Firebase	PP1
P1 (Essential)	Dashboard reads Firebase: current + history + predictions	PP1
P2 (Important)	UDP gateway ingestion path	PP2
P2 (Important)	User Interface	PP2
P2 (Enhancement)	Advanced alert rules and analytics using predicted outputs	PP2

3. SPECIFIC REQUIREMENTS

3.1 External interface requirements

3.1.1 User Interfaces

Key UI screens and elements are specified below.

Screen	Main Elements	Purpose
Overview	Tank cards, last update, connectivity badge, prediction summary	At-a-glance system status
Tank Detail	Charts: pH/EC/temp/humidity/water level + prediction trend	Detailed monitoring and analysis
Comparison	Aligned organic vs inorganic charts on same time axis	Research comparison
Predictions/Warnings	Battery trend, recommended sleep interval, stability score, warnings	Decision support
Admin/Settings	Thresholds, tank mapping, gateway settings (optional)	Site adaptation and control

3.1.2 Hardware Interfaces

Component	Interface	Signals/Data	Notes
ESP32	Wi-Fi + GPIO/ADC/I2C/UART	Sensor values and optional UDP send	Direct cloud upload or UDP mode
Sensors (pH, EC, etc.)	ADC/I2C/GPIO	Environmental/solution metrics	Read frequency depends on project

Wi-Fi Router	802.11	LAN + Internet connectivity	Required for all modes
Gateway PC (optional)	Ethernet/Wi-Fi	UDP receive + cloud upload	Runs udp_gateway.py

3.1.3 Software Interfaces

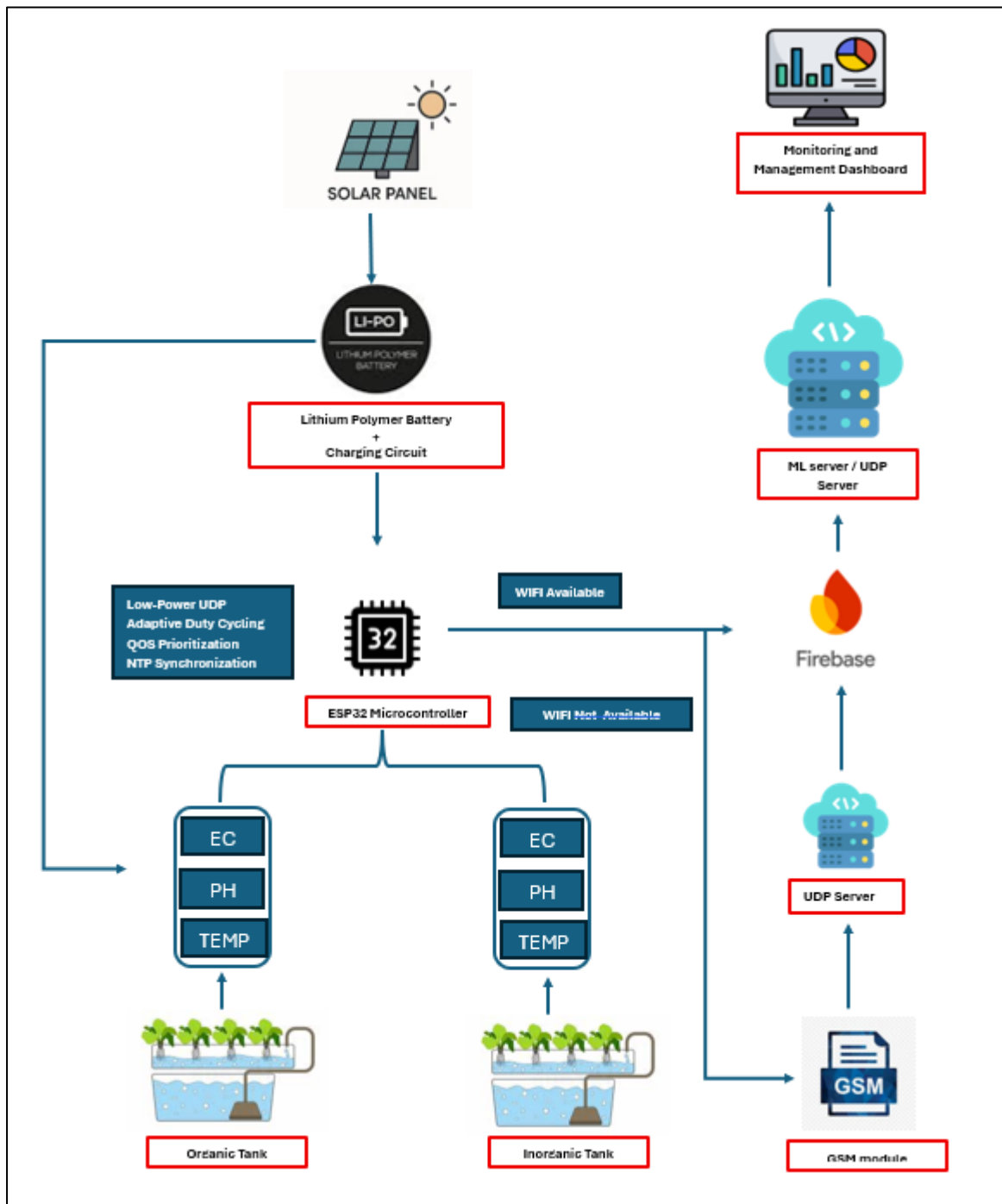
Software Component	Interface	Purpose
ESP32 firmware	Firebase SDK/REST (HTTPS) OR UDP socket	Publish sensor data
udp_gateway.py (optional)	UDP socket + Firebase Admin SDK	Decode + CRC verify + upload
Flask ML server	Firebase Admin SDK + REST API endpoints	Predict and write back
Postman	HTTP client	Test /health and /predict endpoints
Dashboard	Firebase client SDK	Realtime view and history

3.1.4 Communication Interfaces

Two ingestion paths are supported. In direct mode, ESP32 uploads JSON values to Firebase using HTTPS. In gateway mode, ESP32 sends a compact UDP packet to a LAN gateway which validates CRC and uploads to Firebase using Admin SDK. After data is in Firebase, ML and UI processing are identical.

3.2 Architectural Design

3.2.1 High-Level Architectural Design



Primary (cloud-first): ESP32 publishes sensor values to Firebase RTDB. The ML server reads the latest values from Firebase, runs predictions using trained model artifacts, and writes predicted outputs in the dashboard. The dashboard reads for live values, history, and predictions. Optional (gateway-assisted): ESP32 sends a compact UDP packet to a local gateway on the same LAN. The gateway verifies data integrity using CRC, converts the

packet into structured values, and uploads the data to Firebase. This path offloads HTTPS/auth/retry logic from the ESP32 and supports the energy-aware justification.

3.2.2 Hardware and Software Requirements with Justification

Requirement	Chosen Technology	Justification
Central data hub	Firebase RTDB	Realtime cloud storage with minimal backend management; easy UI integration
ML execution	Flask ML server + saved artifacts	Keeps heavy ML off ESP32; reusable API endpoints for testing/integration
Direct ingestion	ESP32 -> Firebase via HTTPS	Simpler deployment and demo; fewer moving parts when internet is stable
Gateway ingestion (optional)	ESP32 -> UDP -> Gateway -> Firebase	Offloads cloud complexity from ESP32; reduces payload overhead and supports energy-efficiency claim
API testing	Postman	Validates endpoints and outputs before UI integration

3.2.3 Risk Mitigation Plan with Alternative Solutions

Risk	Impact	Mitigation	Alternative Solution
Direct Firebase upload failures (weak internet)	Missing data, dashboard not updated	Use optional UDP gateway to offload/retry from	Full custom backend with local DB

		PC; implement backoff	
Gateway offline (if used)	UDP ingestion unavailable	Fallback to direct ESP32 -> Firebase mode	Redundant gateway device
Data corruption in UDP packets	Wrong values stored	CRC verification + discard invalid packets	Use TCP (higher overhead)
Model input mismatch / wrong feature order	Invalid predictions	Strict feature schema + validation in ML server	Retrain models with simplified features
Firebase misconfiguration / access rules	Read/write failures	Use service account Admin SDK for server; test rules	Alternate DB (MongoDB/MySQL)

3.2.4 Cost-Benefit Analysis for the Proposed Solution

- Using Firebase reduces the operational cost of hosting and maintaining a database server.
- The optional UDP gateway adds an extra device (PC) but can reduce ESP32 complexity and communication overhead. ML inference is performed on the server side, requiring compute resources, but improves decision support without increasing node power use.
- The system uses low-cost sensors and an ESP32 microcontroller, reducing initial hardware investment.
- Continuous monitoring reduces the need for frequent manual testing and observation

3.3 Performance Requirements

Performance targets to be validated during evaluation (PP2 refinement):

Metric	Target	Verification
--------	--------	--------------

Firestore update interval	Configurable (e.g., 5–60 min) depending on energy mode	Timestamps stored in RTDB
Prediction refresh time	< 5 s after new reading is available (target)	Server logs and RTDB timestamps
UDP gateway integrity	0 corrupted packets stored (CRC enforced)	Gateway logs + RTDB audit
API response time	< 2 s per prediction request (target)	Postman tests + server logs

3.4 Design Constraints

- ESP32 must keep cloud payloads small and avoid frequent HTTPS handshakes where possible.
- Gateway mode requires ESP32 and gateway to be on the same LAN and the gateway to have internet access for Firestore.
- ML models are loaded from artifact files; changes in feature schema require re-training or re-export.
- Security rules must prevent unauthorized writes to Firestore paths used by sensors and predictions.

3.5 Software System Attributes

3.5.1 Reliability

Reliability is achieved through centralized storage in Firestore with consistent data schemas. In gateway mode, CRC validation prevents corrupted packets from being stored. For direct uploads, retry/backoff logic is applied in ESP32 firmware to handle temporary cloud failures.

3.5.2 Availability

Availability is improved by using two ingestion paths. If the direct cloud path is unstable, gateway ingestion can be used. If the gateway is offline, the system can operate in direct mode. Firestore provides high availability for data access.

3.5.3 Security

Security controls include: HTTPS for cloud communication, Firebase access rules for database paths, and service-account-based access for the ML server. The UDP gateway is restricted to LAN and uses CRC for integrity; additional application-layer authentication can be added if required.

3.5.4 Maintainability

Maintainability is supported by separating responsibilities: ESP32 firmware for sensing/publishing, UDP gateway for ingestion (optional), Flask services for ML inference, and Firebase as the shared hub. Configuration-driven endpoints and clear folder structure (routes/services/artifacts) support updates and debugging.

3.6 Other Requirements

Functional requirements

FR ID	Requirement	Priority
FR-01	ESP32 shall publish latest sensor readings for each tank to Firebase RTDB under defined paths.	Essential
FR-02	ESP32 shall append selected readings to a history node for trend analysis (configurable).	Essential
FR-03	The Flask ML server shall read latest values from Firebase and run battery prediction.	Essential
FR-04	The Flask ML server shall run sleep interval prediction and write recommendation to Firebase.	Essential

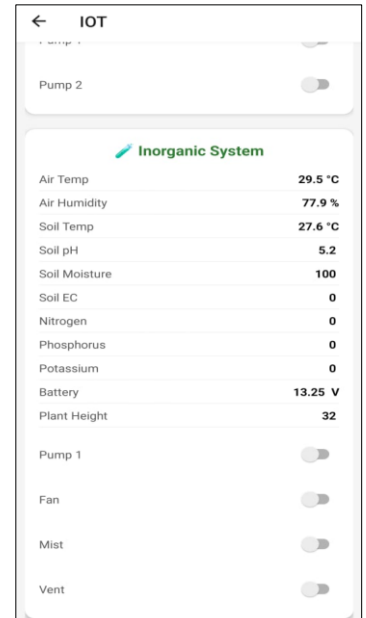
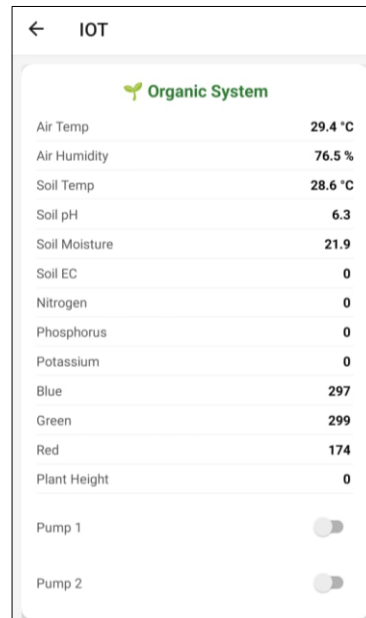
Non-functional requirements

NFR ID	Requirement	Verification
NFR-01 (Energy)	Communication overhead on ESP32 should be minimized; gateway mode should reduce ESP32 workload.	Compare CPU/RAM + power profile where possible
NFR-02 (Integrity)	No corrupted UDP packets should be stored; CRC validation must be enforced.	Gateway logs + RTDB audit
NFR-03 (Usability)	UI must clearly separate Organic vs Inorganic views and highlight predictions/warnings.	UI walkthrough against rubric
NFR-04 (Performance)	ML inference should respond within acceptable time to keep dashboard updates fresh.	Postman timing + server logs

4. SUPPORTING INFORMATION

4.1 Appendices

Appendix A: Sample User Interface Screens shot



Appendix B: Inorganic NFT hydroponic system



Appendix B: Organic NFT hydroponic system



Appendix B: Test Results (Firebase data logs)

Firestore

Project Overview

Project shortcuts

- Realtime Database
- Realtime Analytics
- App Check
- Storage

Product categories

- Build
- Run
- Analytics
- AI

Enable Multi-factor Authentication (MFA) on your Google Account accessing Firebase. [Learn more](#)

Realtime Database

Hydroponic

Data Rules Backups Usage Extensions

Protect your Realtime Database resources from abuse, such as billing or phishing

CD <https://hydroponic-3ddcf-default-rtdb.firebaseio.com>

Your security rules are defined as public, so anyone can steal database

<https://hydroponic-3ddcf-default-rtdb.firebaseio.com>

```
└─ Inorganic
  └─ Organic
    └─ data
      ├── airHum: 76.5
      ├── airTemp: 29.4
      ├── blue: 297
      ├── distance: 0
      ├── gas: 132
      ├── green: 299
      ├── nitrogen: 0
      ├── phosphorus: 0
      ├── potassium: 0
      ├── red: 174
      ├── soilEC: 0
      ├── soilMoist: 21.9
      ├── soilPH: 6.3
      └── soilTemp: 28.6
    └─ sensor
      └─ sensor
```

<https://hydroponic-3ddcf-default-rtdb.firebaseio.com/>

```
└─ Inorganic
  └─ Organic
    └─ data
      ├── airHum: 76.5
      ├── airTemp: 29.4
      ├── blue: 297
      ├── distance: 0
      ├── gas: 132
      ├── green: 299
      ├── nitrogen: 0
      ├── phosphorus: 0
      ├── potassium: 0
      ├── red: 174
      ├── soilEC: 0
      ├── soilMoist: 21.9
      ├── soilPH: 6.3
      └── soilTemp: 28.6
    └─ sensor
      ├── pump1: 0
      └── pump2: 0
```

Appendix C – Hardware Components



5. REFERENCES

- [1] H. S. Sishodia, M. L. Singh, and S. Kumar, “IoT-based Smart Hydroponics System,” in *Proc. Int. Conf. Adv. Comput. Commun. Eng.*, 2021, pp. 1–5.
- [2] R. A. F., A. W. C., L. L. S., I. R. G., L. I. G., I. C., and B. A. S., “Effects of IoT Communication Protocols for Precision Agriculture in Outdoor Environments,” *ResearchGate*, 2024.
- [3] A. Kaur and T. N. Swamy, *Energy-Efficient IoT Systems: Integrating Low-Power Protocols and Adaptive Algorithms for a Greener Future*. ResearchGate, 2024.
- [4] D. van Leemput, J. Famaey, J. Hoebeke, and E. de Poorter, “Energy-aware adaptive scheduling for battery-less 6TiSCH routers in industrial wireless sensor networks,” *IEEE Access*, vol. 12, pp. 180034–180047, 2024.
- [5] M. E. Yuksel, “Power consumption analysis of a Wi-Fi-based IoT device,” *Istanbul Univ. J. Electr. Electron. Eng.*, 2020.